



Race Strategy Reinforcement Learning: Optimising Pitstop Strategy with Emergent Tactics in Formula One

Devin Thomas^{1,2} · Junqi Jiang¹ · Avinash Kori¹ · Aaron Russo² · Steffen Winkler² · Stuart Sale² · Joseph McMillan² · Francesco Belardinelli¹ · Antonio Rago^{1,3}

Received: 3 April 2025 / Revised: 23 April 2026 / Accepted: 18 May 2026 /

Published online: 9 June 2026

© The Author(s) 2026

Abstract

In Formula One, often described as the pinnacle of motorsport, teams compete to design and produce the fastest cars, driven by some of the best drivers in the world, in order to win races. However, a team has little chance of success without effective *race strategy*, i.e. selecting which *tyre compounds* to use and when to take *pitstops* to change between them. Teams' methods for solving this problem are usually limited to linear optimisation, while some run Monte Carlo simulations in simple, best-case situations; these approaches thus fail to take into account the complex interactions between teams' strategies and tactics in this unpredictable multi-agent environment. Further, there is low uptake of AI in this domain, potentially due to a lack of trust in these “black-box” models. In this work, we enable the massive potential of reinforcement learning (RL) models in this space using post-hoc techniques from explainable AI. Specifically, we introduce *Race Strategy Reinforcement Learning* (RSRL), an RL model which allows us to control the strategies of cars in race simulations, with explanations for their actions to help foster trust in users. We first demonstrate that RSRL outperforms baselines of hard-coded and Monte-Carlo strategies, presenting opportunities for improving race strategy for all Formula One teams, and potentially beyond, especially in other areas of motorsport. Next, we analyse RSRL's generalisability to unseen tracks and show how performance on one or multiple tracks can be prioritised via training. We then exhibit the fidelity and comprehensibility of the deployed explanations towards improving user trust in RSRL's decisions. Finally, we highlight the *emergent tactics*, i.e. emergent behaviours representing real-world tactics, learnt by RSRL, pointing towards the general applicability of RL for modelling and even influencing race strategy in Formula One.

Editors: Riccardo Guidotti, Anna Monreale, Dino Pedreschi.

✉ Antonio Rago
antonio.rago@kcl.ac.uk

¹ Department of Computing, Imperial College London, London, UK

² Mercedes-AMG PETRONAS Formula One Team, Brackley, UK

³ Department of Informatics, King's College London, London, UK

Keywords Reinforcement learning · Explainable AI · Formula one race strategy

1 Introduction

Formula One (F1) is a class of motorsport, often described as its pinnacle, where competing teams will spend hundreds of millions of pounds annually in an attempt to win races and, simultaneously, championships.¹ These teams are thus in constant pursuit of the slightest gains in time, achieved by recruiting the best drivers and making their cars faster through ground-breaking engineering. This means that F1 is a hotbed for innovations, with technologies introduced in F1 often percolating down to the wider automotive industry, such as those concerned with sustainability, e.g. regenerative braking systems (Metz, 2013).

Once a race begins, however, teams are unable to make changes to their cars or drivers. They are thus limited to optimising their *race strategy*, which primarily concerns selecting which *tyre compounds* to use and when to take *pitstops*, i.e. leaving the race to change tyres. Tyres *degrade* and become slower as they are used and the different tyre compounds provide a trade-off between speed and rate of tyre degradation. Obtaining the optimal race strategies is far from simple, however, given that live race conditions, e.g. the race strategies taken by other drivers, must be taken into account. This is further complicated by the fact that teams will use *race tactics* to take advantage of temporary differences in the speed in tyres, especially on tracks where overtaking is difficult. Race strategy has thus become a critical factor in determining the cars' finishing positions. This has been accentuated by F1's move to a single tyre manufacturer for all teams, rather than multiple competing tyre manufacturers, which has levelled the playing field between teams and increased the importance of their strategic decisions.² Given the fine margins between cars' speeds, often down to thousands of a second, if a team does not execute a good race strategy, it will have very little chance of success.

Usually, teams decide on candidate strategies before a race, with the help of linear optimisation, with some limited accounting for live situations which may occur such as *safety cars*, i.e. periods where the cars must drive slower than usual due to unsafe conditions on the track. They then run a large number of Monte Carlo simulations to test these different candidate strategies and gain an understanding how well they perform.^{3,4,5} However, this method is time-consuming, computationally expensive, and mentally laborious since teams must first decide on the candidate strategies and then wait for the simulation results to be computed. Additionally, since these strategies are pre-defined, they are unable to dynamically account for live race situations, giving only general guidance for the race under normal

¹ <https://www.forbes.com/sites/csylt/2020/01/14/red-bull-reveals-how-much-it-really-costs-to-run-an-fl-team/>.

² <https://www.autosport.com/fl/news/why-tyre-wars-have-largely-become-a-thing-of-the-past-in-motorsport/10526870/>.

³ <https://www.oracle.com/customers/red-bull-racing-case-study/>.

⁴ <https://www.mercedesamgf1.com/news/web-exclusive-q-and-a-with-rosie-wait..>

⁵ <https://www.autosport.com/fl/news/how-red-bull-comes-up-with-the-perfect-strategy-for-verstappen-and-perez/10619624/>.

conditions and no instruction for deploying, or defending against, race tactics. Further, the methods for obtaining the optimal strategies do not readily explain their outputs, meaning the race strategists' trust in them will be limited.

- **Contributions.** In this work, we deploy reinforcement learning (RL) models in F1 race strategy, capitalising on their potential in real-time strategy applications, such as *Go* (Silver et al., 2016), *Atari* games (Mnih et al., 2015), *Starcraft* (Vinyals et al., 2019) and *Dota 2* (Berner et al., 2019). We adapt and deploy an RL architecture for this task, demonstrating our approach's potential with a Monte Carlo race simulator of the Mercedes-AMG PETRONAS Formula One Team, both in optimising race strategy and modelling race tactics used in the real world, even when minimal expert knowledge is provided. We then deploy post-hoc explainable AI (XAI) techniques (see (Ali et al., 2023) for a recent overview), to provide reasoning for the optimised strategies and modelled race tactics towards improving race strategists' trust in the models. Concretely, we make the following contributions⁶We introduce a flexible, extensible and portable architecture for real-time application of RL to F1 race strategy, allowing for training and deployment with different data sources, including the *F1 23* video game.
- We examine the potential of four RL architectures, compared via Monte Carlo simulations to identify that with the best average finishing position, which is then put through extensive hyperparameter tuning.
- We thus present *Race Strategy Reinforcement Learning* (RSRL), utilising the best-performing RL architecture, which outperforms baseline models in selecting race strategies for the 2023 Bahrain Grand Prix (GP).
- In a generalisability study, we show how RSRL's performance over one track or multiple tracks can be prioritised via training, demonstrating further improvements in performance over the baselines.
- We supplement RSRL with three post-hoc XAI techniques to explain its decisions, both in simulations and live races, giving informal examinations of their fidelity and comprehensibility.
- We provide a detailed case study of the 2023 Bahrain GP, showing how not only does RSRL replicate real-world strategic decisions made by strategists in live races, but it also exhibits complex race tactics as emergent behaviours.

Paper Structure. After covering the related work (§2), we first introduce our approach (§3). We then cover its extensive evaluation (§4) and the results from the XAI deployment (§5), before showcasing RSRL's behaviour in a case study (§6). Finally, we conclude and discuss future work (§7).

This paper builds on and extends (Thomas et al., 2025) in a number of ways. We have made substantial additions to §3 and §4: §4.1 is entirely new, and additional details and results have been included throughout these sections. Next, §6 is a completely new section, dedicated to a case study that highlights the emergent tactics of RSRL, which were not considered in (Thomas et al., 2025). Further, Appendix A gives additional experimental details.

⁶Our code is publicly available at <https://github.com/DevinThomas185/F1-RSRL/>.

Finally, we have revised and refined much of the paper, adding description and details to better convey both the motivations behind and the potential of our approach.

2 Related Work

RL in Motorsport. Many motorsport categories other than F1, such as Formula E, WEC, Indycar and GT Racing, also require effective race strategies. Within Formula E, Liu and Fotouhi (2020) and Liu et al. (2021) explore race strategy using *distributed deep deterministic policy gradients*, neural networks and *Monte Carlo Tree Search*. In GT Racing, Boettinger and Klotz (2023) implement a Deep Q-Network (DQN) to optimise strategy decisions over a limited action space. They find that a baseline Q-Learning model is significantly beaten by the DQN model. Meanwhile, Wurman et al. (2022) employ a *quantile regression soft actor-critic* model to drive GT cars in the *Gran Turismo* game. Their model outperformed the world's top drivers and demonstrated driving strategies such as different corner overtakes, highlighting its ability to learn human behaviours. In F1, Heilmeier et al. (2020) introduce a *virtual strategy engineer* comprising two separate neural networks that predict whether or not a pitstop is taken based on given race inputs. They discuss as future work the possibility of taking an RL-based approach, as we introduce in this paper. Piccinotti et al. (2021) use open-loop planning with Monte Carlo sampling and Temporal Difference to identify an optimal race strategy, evaluating different planning algorithms on the simulator defined by Heilmeier et al. (2020). Finally, Heine and Thraves (2023) explore race strategy through dynamic programming (DP) and later incorporate random events such as safety car periods through stochastic DP. They find that both models can be solved to optimality. The DP approach allows the answering of questions such as "If there is a safety car period in the current lap, is it worth making a pitstop?" and "If yes, which tyre compound should we change to?". Comparatively, the stochastic DP approach considers delaying pitstops in favour of benefiting from future safety car periods, which improves upon the DP model when randomness is increased during simulations. Away from motorsport, there has been growing interest in applying machine learning in other sports, e.g. basketball (Zimmermann, 2023) and football (Wang et al., 2024).

However, none of the works mentioned here, in F1 or otherwise, apply RL, coupled with post-hoc explainability techniques, to optimise race strategy.

XAI for RL. XAI has gained prominence due to the growing role of AI in society, and thus, it is increasingly important that humans comprehend the predictions made by these systems. Milani et al. (2024) and Bekkemoen (2024) give recent reviews of different explainability techniques and give taxonomies thereof. Methods such as *DAGGER* (Ross et al., 2011), *VIPER* (Bastani et al., 2018), *PIRL* (Verma et al., 2018), and *AI-Interpret* (Skirzynski et al., 2021) convert RL policies into interpretable formats such as decision trees, whereas reward decomposition (Van Seijen et al., 2017) is used to decompose the rewards into a set of additive terms with semantic meaning. Some methods directly generate explanations. For example, natural language explanations employed by Hayes and Shah (2017) utilise templates for the agent to complete. *Saliency maps* are also used by Greydanus et al. (2018), though they have been considered insufficient for explaining RL due to the subjective conclusions that can be drawn from their explanations (Milani et al., 2024).

As for the methods we use in this paper, we first note that *SHAP* (Lundberg and Lee, 2017) is a method to explain individual predictions based on *Shapley values* (Shapley, 1953). *SHAP* explains the prediction of an instance by computing the contribution of each feature to the prediction using computed Shapley values. *TimeSHAP* was then introduced by Bento et al. (2021) and extends the use of Shapley values into sequential domains. *TimeSHAP* is a model-agnostic recurrent explainer that computes feature-level attributions, highlighting the importance of features of the current time step to a prediction.

Another method we use is *VIPER* (Bastani et al., 2018), an *imitation learning* method that builds upon the data aggregation method of *DAGGER* (Ross et al., 2011). As a surrogate model, *VIPER* builds decision trees that closely mimic the behaviour of a pre-trained oracle. *VIPER* leverages the fact that the cumulative reward of each state-action pair is provided along with the optimal action, thus producing decision tree policies that are an order of magnitude smaller than those learnt by *DAGGER*.

Finally, Carreira-Perpiñán and Hada (2021) generate counterfactuals for decision trees by constructing a linear programming problem, selecting the leaf node with the least distance to the current data point. For the distance function, the ℓ_2 distance can be used to encourage changing many features minimally, whilst the ℓ_1 distance encourages fewer of the features to be changed by a greater magnitude.

To exhibit a broad range of XAI approaches in this setting, we use *TimeSHAP*, *VIPER* and decision tree counterfactuals in this paper, and leave other deployments of XAI to future work.

3 Race Strategy Reinforcement Learning

In this section, we introduce our approach, detailing how we formalise the RL problem (§3.1) in which we will deploy various RL models via our system architecture (§3.2).

3.1 Problem Formalisation

State Space. Selecting the state space, $\mathcal{S}$, is critical to ensure that the RL model can execute successful strategies without being overloaded with features. The chosen features are outlined in Table 1. It should be noted that there are three tyre compounds used: soft, medium and hard. The soft tyres are generally the fastest, but they degrade and slow down at a faster rate than the other tyres. Meanwhile, the hard tyres are longer-lasting, but can be comparatively slower even when new, whilst the medium tyres give a compromise between the two.⁷ Fig. 1 demonstrates one example set of multi-linear *tyre curves*, which characterise the lap time lost (relative to a base lap time) due to the degradation experienced as a car drives more laps on the tyre.

The deployment of RL introduces a new challenge in data preprocessing since we do not have a predefined training dataset that dictates the limits of each feature. Theoretically, some features have no limit in their values. For instance, the gap to the leader s_{gl} , i.e. the time in seconds between the leader of the race and RSRL's car, can be arbitrarily large if the driver takes a pitstop every lap, subsequently making them fall further and further back from

⁷ It is necessary to note that the characteristics of each tyre vary between tracks. This can be inferred from the specific tyre curves calculated for each track.

Table 1 The (boolean, categorical, integer and continuous) variables used in the reward function and the state space provided to the RL model, after data normalisation

State space	$\mathcal{S}$	Type	Values
*Terminal	$s_{terminal}$	Bool	True, False
Track	s_{track}	Enum	Track Enumeration
Safety Car	s_{sc}	Cat	Full, Virt., None
Position	s_{pos}	Int	[1,..., 20]
Scaled Position	s_{spos}	Cont	[0, 1]
Race Progress	s_{rp}	Cont	[0, 1]
Current Tyre	s_{tyre}	Cat	S, M, H
Tyre Degradation	s_{td}	Cont	[?, ?]
Scaled Tyre Degradation	s_{std}	Cont	[0, 1]
Soft Available	s_{soft}	Bool	True, False
Medium Available	s_{medium}	Bool	True, False
Hard Available	s_{hard}	Bool	True, False
Gap Ahead	s_{ga}	Cont	[?, ?]
Scaled Gap Ahead	s_{sga}	Cont	[-1, 1]
Gap Behind	s_{gb}	Cont	[?, ?]
Scaled Gap Behind	s_{sgb}	Cont	[-1, 1]
Gap to Leader	s_{gl}	Cont	[?, ?]
Scaled Gap to Leader	s_{sgl}	Cont	[-1, 1]
Last Lap to Reference	s_{llr}	Cont	[0, 2]
Valid Finish	s_{vf}	Bool	True, False

One-hot encoding is used for all Enums and Boolean types. Data normalisation was applied by generating a dataset of simulated values, running many simulations and applying linear scaling with clipping. Features labelled with * are not provided as an input to the model, but are used in the reward function

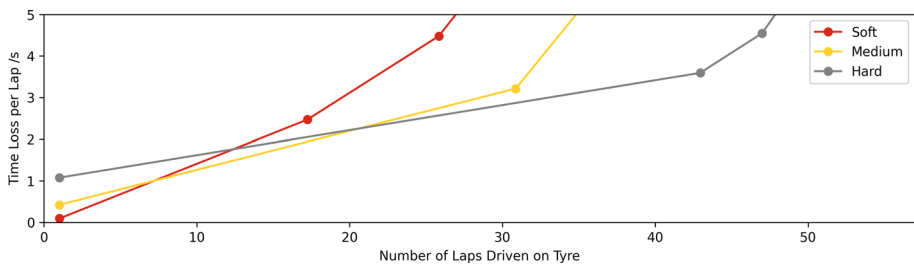


Fig. 1 An example set of tyre curves which characterise the time lost per lap for each tyre compound. Here, the soft tyre is the fastest in the first few laps. However, between laps 8 and 12, it becomes slower than the medium tyres and then the hard tyres. This demonstrates the trade-off between speed and longevity in the tyre selection

the leader. We thus created a dataset with value ranges determined by observations from a reasonable number of simulations to estimate their working ranges. From this set of values, we created custom scaling functions that linearly scale each feature.

The complete set of features is as follows. The terminal feature ($s_{terminal}$) depicts the terminality of the state. The model can also be made aware of the track that it is strategising for (s_{track}), a critical feature for the generalisability study in §4.3. Safety car information (s_{sc}) is also provided to the model so that it may be able to take advantage of the reduced loss of time taken to make a pitstop under a safety car. The current position (s_{pos}) is linearly scaled (s_{spos}) to inform the model of its general performance, whilst the percentage of race completion (s_{rp}) informs its distance to simulation completion. The current compound

(s_{tyre}) is another crucial feature, along with the current tyre's degradation (s_{td} , and its scaled version s_{std}) according to the tyre curve, and whether there are sets remaining for each of the tyre compounds (s_{soft} , s_{medium} , s_{hard}). We also inform the model of the gap to the cars positionally⁸ ahead (s_{ga} , and its scaled version s_{sga}), positionally behind (s_{gb} , and its scaled version s_{sgb}), and the car in the lead (s_{gl} , and its scaled version s_{sgl}). To inform the model of its pace, we include how fast or slow its last lap time was with respect to a reference lap time (s_{llr}). Finally, a flag (s_{vf}) informs the model if it can legally finish the race given the current race state to guide it towards following the race rules and regulations.

Action Space. At each lap, RSRL decides whether to stay out, or to execute a pitstop to change tyres to soft, medium or hard tyres. Subsequently, the action space at any timestep, t , consists of four actions, $a^t \in \{\text{no pit, pit soft, pit medium, pit hard}\}$, representing taking no pitstop or taking a pitstop to one of the three tyre compounds, respectively. The tyre on which the model begins the race is chosen by the Monte Carlo race simulator. Furthermore, the number of times each action can be selected during a race simulation is limited, since the available number of each tyre compound is specified in the F1 regulations. For example, the maximum number of times a *pit hard* decision can be taken during a race is two, since teams are allocated only two hard tyre sets during a race. Similarly, there are eight soft sets and three medium sets allocated.⁹

Reward Function. Given that our goal in this paper was to minimise the expert knowledge which we provide to RSRL, we designed a reward function based on the rules and regulations of F1 races, e.g. heavily penalising runs which do not include a pitstop. The reward function at any timestep t , R^t is thus defined as:

$$R^t = \begin{cases} -1000 & \text{if } a^t = \text{pit } x \wedge \neg s_x^t \text{ for } x \in \{\text{soft, medium, hard}\} \\ -1000 & \text{if } s_{terminal}^t \wedge \neg s_{vf}^t \\ -10 & \text{if } a^t \neq \text{no pit} \wedge s_{vf}^t \\ 100 * P[s_{pos}^t - 1] & \text{if } s_{pos}^t \leq 10 \wedge s_{terminal}^t \\ 0 & \text{if } s_{pos}^t > 10 \wedge s_{terminal}^t \\ 1 & \text{otherwise} \end{cases}$$

where $P = [25, 18, 15, 12, 10, 8, 6, 4, 2, 1]$.

Rewards of 1 are given for steps in which the model progresses to the next lap. This can be done by not pitting, or by taking the first valid pitstop (i.e. a pitstop to a different tyre, a requirement in F1). A penalty of -10 is given for additional pitstops beyond the first valid pitstop. A terminal reward is also given based on the final finishing position which equates to 100 times the points the driver would receive in F1 for finishing in that position, where the points for first to tenth place, denoted P1 to P10, are 25, 18, 15, 12, 10, 8, 6, 4, 2, 1, respectively, and 0 are given for P11 to P20, i.e. eleventh to last place. For example, a P1 finish rewards 2500 points. By giving a reward of 1 for normal steps, the model maximises its total reward by ensuring it steps through to the end. When the model fails a simulation,

⁸“Positionally” refers to the cars ahead and behind in the current race order. We disregard the actual cars ahead and behind on track, since they may be lapped cars (i.e. cars over a full track length behind), which must allow overtaking cars to pass when they are being lapped.

⁹It is important to note that during a GP weekend there are four other sessions, such as practice and qualifying, for which, together with the race, these tyre sets are allocated. Therefore, the available tyre sets used in the race are fewer than those allocated for the whole GP weekend.

such as taking a pitstop to a tyre that is not available, it is penalised heavily with -1000 , as in (Boettinger and Klotz, 2023). Reward shaping is not used in the reward function because it is inherently difficult in this context. This is because we are unable to determine midway through a race whether a decision was good or not. Instead, we only find out if a strategy was successful at the end of a race when we discover the final finishing position. It should be noted that although only one pitstop is required by the regulations, a single-pitstop strategy is not necessarily the fastest, and so the optimal number of pitstops is highly dependent on many factors in the state space $\mathcal{S}$.

3.2 System Architecture

The design of the system architecture lends itself well to many features and is illustrated in Fig. 2.

Mercedes' Monte Carlo Race Simulator. Central to the training loop is the black-box Monte Carlo race simulator, provided by Mercedes. The race simulator allows us to simulate any race from the 2023 season, with the ability to pause the simulation lap-by-lap. This feature presents the opportunity for an RL model to control the decisions of a chosen car.

Race Simulator Interface. Since the Mercedes Race Simulator handles proprietary data definitions, it was important to abstract our state and action space away from these definitions. By defining custom `UnifiedRaceState` and `UnifiedRaceStrategy` classes with a translator layer in-between, the model maintains its interoperability with the race simulator and decouples RSRL from the black-box. In implementing the Race Simulator Translator through the `Race Simulator Interface`, we improve portability, allowing for the use of different data sources such as live race data or the *F1 23* video game. By implementing the interface through another translator to convert live race data into the `UnifiedRaceState`, the model can be trained using the Monte Carlo race simulator, but deployed during a race. Such a method is currently not employed in industry, which defaults to hard-coded race strategies. Portability with different data sources also enables the ability to refine the model using different information. For example, the model could be fine-tuned using a real-world Driver-In-The-Loop Simulator to produce feasible strategies depending on a

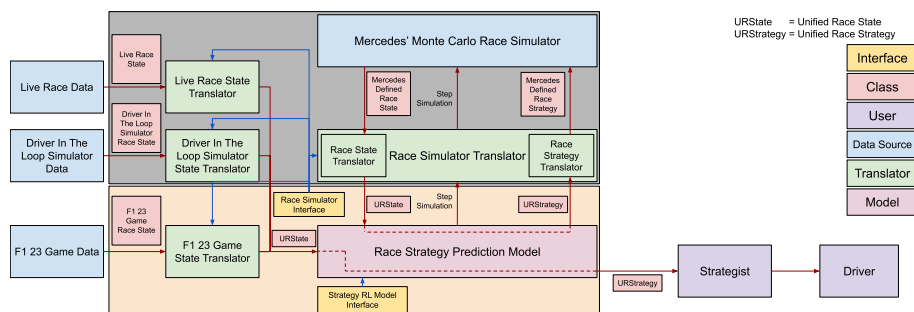


Fig. 2 The system architecture implemented utilises abstraction to provide future flexibility, extensibility and portability. It allows for the substitution of different data sources for model prediction and the modification of race states and strategies

driver's racing style. The definition of the `UnifiedRaceStrategy` class also allows us to flexibly change the definition of a "race strategy" in the future, without significantly changing existing components. This could include the addition of other variables, e.g. target lap times or engine modes. Implementing the RL model using an interface, similarly, allows the substitution of different model architectures and encourages the exploration of different approaches to find the best solution for each application of our approach.

Race Strategy Prediction Model. In order to demonstrate how simulation runs take place, we let $\mathcal{S}$ denote, as before, the state space that the `UnifiedRaceState` is sampled from, i.e. that which is used by the RL model, and $\mathcal{S}'$ be the Mercedes race simulator state space (which remains confidential). Although both state spaces describe the underlying race, $\mathcal{S}$ is an abstracted representation of $\mathcal{S}'$, constructed to provide the modular structure as depicted in Fig. 2. We define the state translation function $f : \mathcal{S}' \rightarrow \mathcal{S}$, which maps the Mercedes race simulator state to the `UnifiedRaceState`, i.e. $s^t = f(s^{t'})$ for any timestep t . Similarly, we define an action translation function $g : \mathcal{A} \rightarrow \mathcal{A}'$ where $\mathcal{A} = \{\text{no pit, pit soft, pit medium, pit hard}\}$ is RSRL's action space and $\mathcal{A}'$ is the (confidential) Mercedes race simulator action space. Then, at any timestep t , g converts the chosen action in `UnifiedRaceStrategy` to that in the Mercedes race simulator. Since simulations are seeded, each run will be unique, giving varying values between simulations for the state variables in $\mathcal{S}$, e.g. the starting positions, starting tyres and gap to leader. The simulation parameters which remain constant are variables in $\mathcal{S}'$ such as expected driver pace, tyre curves or overtaking parameters, the exact values for which remain confidential. In any single training run at any timestep t , RSRL is provided with the state $s^t \in \mathcal{S}$, where $s^t = f(s^{t'})$. RSRL then selects action a^t , which is passed through the translation layer to give $a^{t'} = g(a^t)$. This action is executed in the Mercedes race simulator producing state $s^{t+1'}$, which is translated back to $s^{t+1} = f(s^{t+1'})$. These parameters are passed to the reward function, $R^t(s^t, a^t, s^{t+1})$ to produce the reward. RSRL uses this reward to update its policy, before selecting its next action a^{t+1} , which is passed back through translator layer to give $a^{t+1'} = g(a^{t+1})$ and executed in the Mercedes race simulator, producing state $s^{t+1'}$. The loop ends when the model has either encountered an error, a fault or completes the simulation, at which point it receives its terminal reward.

In developing the model, we considered various architectures. The first model, the original DQN (Mnih et al., 2013) was motivated by the approach of Boettinger and Klotz (2023). The Constrained DQN (CDQN) (Kalweit et al., 2020), which allows for constraints to be incorporated in the learning, was also tested in an attempt to add the hard constraint of only allowing *available* tyres to be selected. However, neither of these models effectively capture the nature of partial observability, whereby the agent only receives partial observations of the underlying state space. Consequently, we decided to utilise the Deep Recurrent Q-Network (DRQN) architecture as outlined by Hausknecht and Stone (2015). We motivate this architecture through its ability to capture temporal dynamics which are important in this setting. For example, the model needs to understand whether it is catching the car ahead or falling behind it. This same issue motivated Heilmeier et al. (2020) to introduce new features representing these dynamics. Finally, due to the limitation of simulation speed, the Asynchronous Advantage Actor-Critic (A3C) model (Mnih et al., 2016) was also considered for its ability to train on more data in parallel. All model architecture details can be found in the *Models* directory in the codebase (see Footnote 4).

4 Evaluation

In this section, we conduct a comprehensive evaluation of RSRL. First, we report details of our model selection and findings during hyperparameter tuning, showing how our best model was obtained (§4.1). Then, we assess RSRL's performance against existing baselines (§4.2). Finally, we analyse how well it generalises to unseen tracks (§4.3).

4.1 Training Details

Model Selection. We trained four types of RL models (DQN (Mnih et al., 2013), CDQN (Kalweit et al., 2020), DRQN (Hausknecht and Stone, 2015) and A3C (Mnih et al., 2016)) for roughly 12,000 random seed simulations with a set of basic hyperparameters and compared their average finishing positions across 30 randomly seeded race simulations of the 2023 Bahrain GP. This race was selected for training and testing of the overall performance of RSRL since the expected strategy is fairly simple, and so was recommended as a good starting point by the Mercedes race strategists (other tracks were used in the generalisability study in §4.3). The parameters of these models are outlined in Appendix A.1 These preliminary models were compared with two different baselines: a *Fixed Strategy* model and a *Mercedes Probabilistic* model. The Fixed Strategy model employs one of the possible strategies defined on the F1 website for the 2023 Bahrain GP¹⁰, and selects a random lap within the pit window range. For example, if the suggested strategy is Soft-Hard with a pitstop somewhere between laps 15 and 20, then the Fixed Strategy model may choose to take its pitstop on lap 17. The (proprietary and thus confidential) Mercedes Probabilistic model uses simulated conditions with probabilistic events to evaluate and estimate the best tyre strategy. We then further tested the best checkpoint of each model type for another 150 randomly seeded test simulations of the Bahrain GP during which we analysed and compared their behaviours.

The finishing positions of the test simulations are plotted in Fig. 3 with the full results in Appendix A.1. The results show that the best DRQN model is the most performant model alongside the Fixed Strategy model, with these two models being comparable across all metrics (mean, best-case, worst-case, etc.). For example, the 90th percentile and worst-case results for both of these models were P8.00 and P16.00, respectively. Compared with the Mercedes Probabilistic model's P5.86 average finishing position, the DQN model achieves a better average P5.51 finish, whilst CDQN and A3C models demonstrate worse average finishing positions of P6.05 and P8.10, respectively, with varying best- and worst-case performance.

We acknowledge that training RSRL by optimising the expected cumulative reward without penalty on variance or guarantee on worst-case performance is a limitation of our approach. However, it is important to note that doing so would likely converge the model to conform towards a simple strategy which optimises for the tyre curves in a best case situation without the flexibility for considering the actual race conditions, thus causing the model to repeatedly select the strategy that can be precomputed easily beforehand (as in the Fixed Strategy model). The benefits of RSRL are in its decision making under different scenarios (as we will demonstrate in a case study in §6), where variance in strategy selection should be

¹⁰<https://www.formula1.com/en/latest/article/strategy-guide-what-are-the-possible-race-strategies-for-the-2023-bahrain.1nYrqBvGh8pZs9z6anDCdi..>

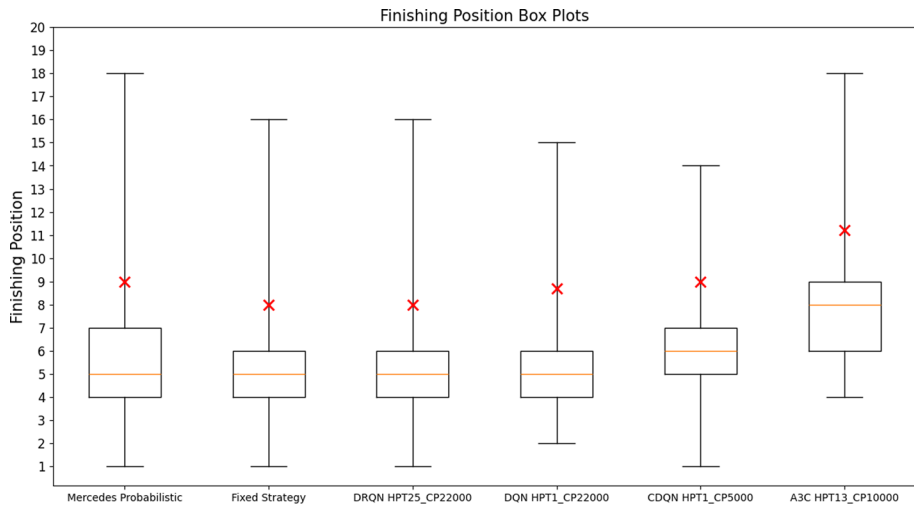


Fig. 3 Box plots (from left to right) for the Mercedes Probabilistic model, the Fixed Strategy model and each of the four preliminary models showing the finishing positions over 200 (randomly seeded) simulations. Red crosses denote the 90th percentiles

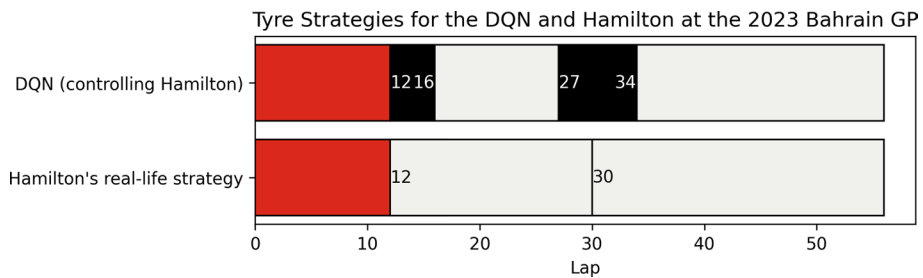


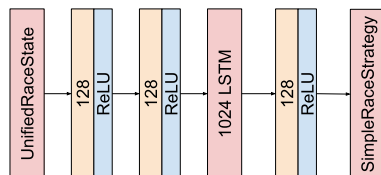
Fig. 4 Comparison of the strategies used by the best performing DQN model and the real-world 2023 Bahrain GP race strategy used by Hamilton, where red indicates soft tyres and white indicates hard tyres. The black sections indicate the range of the laps in which a pitstop was taken. This was calculated by taking all Soft-Hard-Hard strategies and plotting the interquartile range for each pitstop

considered. Further, the model is not intended to serve as a replacement for race strategists, in which case these worst-case results could be catastrophic, but as a tool to support in their decision-making, allowing the race strategists to assess this risk.

We observed that realistic tyre strategies were learnt even by these preliminary models. For example, the most commonly employed strategy by the DQN is illustrated in Fig. 4. Notably, the DQN learns two-stop SHH (Soft-Hard-Hard) strategies with practical pit windows. It can be seen from Fig. 4 that this strategy captures Hamilton's real-world strategy (lap 12 and lap 30) from the same race. Since the DRQN is the most powerful model in our setting, from this point onwards we will focus on the DRQN and omit the other three model types. We also discuss the specific tyre strategies of the DRQN further in §6.

Table 2 Final hyperparameters chosen for RSRL

Hyperparameter	Final value
Epsilon	1
Epsilon decay	0.999
Minimum epsilon	0.005
Gamma	0.99
Learning rate	0.001
Weight decay	0.001
Replay buffer size	1000
Episodes to update target	100

Fig. 5 The final DRQN architecture of RSRL

Hyperparameter Tuning. Extensive hyperparameter tuning was completed for the DRQN¹¹ with different model architectures and training configurations to obtain the best-performing models. Overall, there were 35 hyperparameter tuning experiments, completed manually, each running for 36 h, and completing between 18,000 and 26,000 simulation runs. The resulting 726 models were then tested over 30 random simulations to find the most performant models.

Generally, one would execute hyperparameter tuning using a framework such as Optuna or RayTune. However, this was not feasible due to long training times resulting from the race simulations. It was more convenient to individually submit different training jobs to a HPC cluster for training. Whilst we could use these frameworks for parallel hyperparameter tuning, this would require submitting a single job with larger resource requests instead of multiple smaller resource requests. Initial hyperparameters were selected through educated assumptions and fine-tuned by assessing perturbations to the original set.

Hyperparameter tuning improved the average finishing position of the DRQN from P6.00 to P4.79. The most notable findings are discussed further in Appendix A.2. The final hyperparameters for the selected DRQN model are defined as in Table 2, with the network structure as in Fig. 5.

4.2 Model Performance

To test the performance of a trained RSRL model, we take a fixed set of different random seeds and run identical Monte Carlo simulations for RSRL and with the two baselines from §4.1, and compare their results. To begin with, we start with one track for both training and testing: Bahrain 2023 with a driver with a pace equivalent to an expected finish of P5.5 (i.e. in the third fastest team for Bahrain).¹²

¹¹ Only basic hyperparameter tuning was completed for the other models due to time and computational constraints, before the DRQN was selected as the model with which to proceed. We leave to future work further model exploration through the extensive tuning of the other models: the DQN, CDQN and A3C.

¹² We leave assessing RSRL when trained on different drivers to future work.

Through testing, we found that the best-performing model of RSRL for the 2023 Bahrain GP achieved an average finishing position of P5.33 [95% CI: P5.14, P5.67] over 1920 simulations, whilst the Fixed Strategy and Mercedes Probabilistic model achieved P5.63 [95% CI: P5.22, P6.06] and P5.86 [95% CI: P5.76, P5.95] respectively. We performed Wilcoxon rank-sum tests with Bonferroni correction on the results, showing significant gains for RSRL over the Mercedes Probabilistic model ($U = 173600$, $p = 0.00167$, $p_{adj} = 0.00335$) but not over the Fixed Strategy model ($U = 5820$, $p = 0.170$, $p_{adj} = 0.340$), nor for the Fixed Strategy model over the Mercedes Probabilistic model ($U = 58741.5$, $p = 0.694$, $p_{adj} = 1.000$). It is important to note that whilst this appears to be a small improvement, there is a limit to the performance due to the simulations. For example, it is impossible to consistently finish P1 with the stochasticity of the race simulator and the modelled performance of the car. Improving upon both the Fixed Strategy and Mercedes Probabilistic models thus demonstrates a noteworthy achievement. Figure 6 demonstrates the improved performance of RSRL over the Mercedes Probabilistic and Fixed Strategy models.

4.3 Generalisability

To understand how well RSRL performs when trained across a different number of tracks, we trained three separate instances on 1, 2 and 9 tracks, named RSRL-1, RSRL-2 and RSRL-9, respectively. Guided by expert advice from Mercedes strategists, we selected tracks which exhibit diverse track characteristics, thus aiming to cover a variety of expected strategies, e.g. Azerbaijan, Singapore and Saudi Arabia are street tracks, while Japan, Italy and Spain are traditional tracks. To determine how well these models generalise, we tested each trained model on the tracks that they had and had not seen. Each model was tested approximately

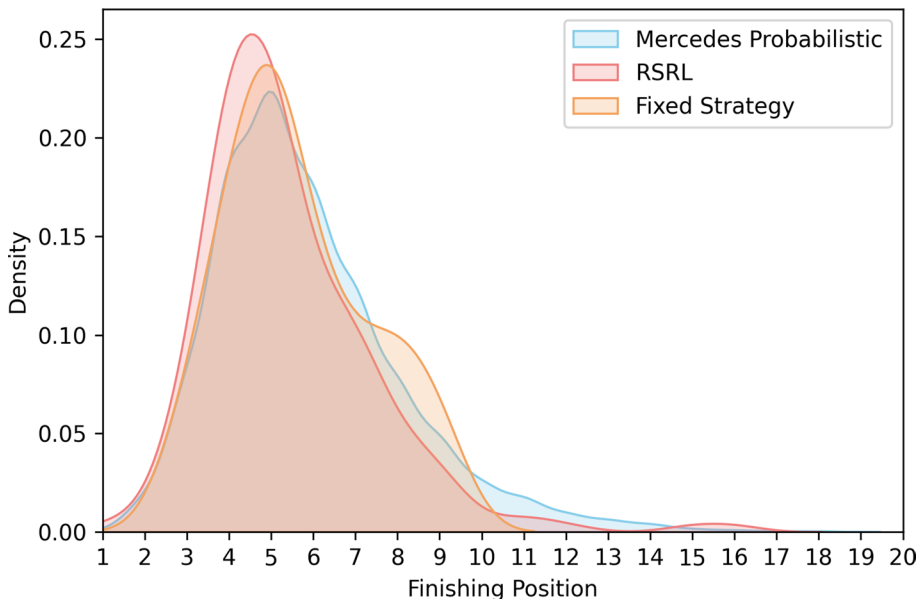


Fig. 6 Density plot of finishing positions for the Mercedes probabilistic model, RSRL and the fixed strategy model

900 times per track. Table 3 shows the average finishing positions, as well as the tracks in each of the models' training sets, shown in underlined font.

Table 4 demonstrates the comparative performance of RSRL-1, RSRL-2 and RSRL-9 against both baseline models. The results suggest that the specialist models, RSRL-1 and RSRL-2, perform better than both baselines on their seen training tracks. It can be seen that as more tracks are added to the training set, the comparative performance on seen tracks worsens (from -1.97 to -0.92 to -0.20 vs. Fixed Strategy, and from -1.77 to -0.93 to $+0.87$ vs. Mercedes Probabilistic). Conversely, as the number of training tracks increases, the comparative performance on unseen tracks improves (from $+3.04$ to $+4.44$ to -0.86 vs. Fixed Strategy, and from $+6.40$ to $+4.01$ to -0.21 vs Mercedes Probabilistic). These results demonstrate how models can be trained to execute different strategies across different tracks, improving the generalisability of the model, but at the cost of performance, when they are trained in the same timeframe.

As a qualitative assessment, we analyse the resulting tyre strategies of the three models and compare them with the tyre strategies taken by the Mercedes Probabilistic model and two of the pre-defined strategies employed by the Fixed Strategy model. Strategies are written in a shortened form where $S[10, 20]M$ indicates a Soft-Medium strategy with the pitstop occurring between laps 10 and 20. In this analysis, we look for reasonable strategies, i.e. those with: (1) between one and three pitstops; (2) a harder compound being used for more laps than a softer compound; and (3) visual similarity to or a better average finishing position (listed in Table 3) than the Mercedes Probabilistic or the Fixed Strategy models.

1-Track Model. For RSRL-1, we note how the strategy taken for Japan (its only training track) is a relatively reasonable $M[16,22]S$ strategy, seen in Fig. 7 though the soft tyre is used for longer than the medium. However, the model successfully employs a one-stop strategy. From Table 3, we see how this model outperforms the Fixed Strategy model, finishing an average of 1.97 positions ahead. The model struggles with the Abu Dhabi and Mexico tracks, where the strategies chosen are not reasonable. For instance, in Abu Dhabi, seen in Fig. 7, the chosen strategy is $M[38,39]S[41]S[42]S$, which finishes on average 8.5 positions behind the Mercedes Probabilistic model.

2-Track Model. RSRL-2 similarly demonstrates reasonable strategies for its training tracks, Japan and Bahrain. Analogously to RSRL-1, it produces mixed results on the unseen tracks, with unreasonable strategies being used in the Mexico race, i.e. $M[3]S[44]H[46]M$, as shown in Fig. 7. Compared to the other models' strategies, RSRL-2 takes too many pitstops, performs the worst of all the models, and finishes on average 9.54 positions behind the Mercedes Probabilistic model.

9-Track Model. RSRL-9 demonstrates reasonable strategies across all of the races. Notably, we see how strategies for the Abu Dhabi, Japan, Mexico and Saudi Arabia races all resemble strategies employed by the Mercedes Probabilistic model and the Fixed Strategies. However, this model demonstrates a very wide window for its pitstop for the Abu Dhabi race. Its lack of precision results in its poorer average finishing position, being 4.01 positions behind the Mercedes Probabilistic model, but still ahead of RSRL-1 and RSRL-2. RSRL-9 is the only model to outperform the Mercedes Probabilistic model in Hungary, selecting a one-stop strategy compared to the Mercedes Probabilistic model's two-stop, and finishes on average 0.1 positions ahead.

Table 3 Average finishing position for each RSRL-X model when trained on X tracks and tested across 14 tracks, with averages over all, seen and unseen tracks, compared with the Fixed Strategy and Mercedes Probabilistic models

Models	Jpn	Bah	Aze	GB	Hun	Ita	Sin	Qat	AD	Spn	SA	Aut	Mex	USA	All	Seen	Unseen
RSRL-1	4.45	9.62	12.33	6.09	12.29	16.25	5.29	8.12	15.75	5.57	6.77	8.29	8.33	6.75	8.99	4.45	9.34
RSRL-2	3.75	6.47	15.33	15.67	8.83	13.50	4.33	18.00	13.75	2.50	9.25	13.60	13.50	12.00	10.75	5.11	10.79
RSRL-9	4.94	<u>6.45</u>	<u>7.36</u>	<u>6.07</u>	4.23	<u>10.96</u>	4.02	<u>3.96</u>	<u>11.26</u>	3.06	6.83	6.06	4.36	2.71	5.88	6.58	4.60
FS	6.42	5.63	9.67	7.07	6.05	7.96	3.60	6.63	7.95	6.65	6.33	6.02	6.07	2.22	6.31	-	-
MP	6.22	5.86	6.95	5.00	4.33	9.01	3.88	2.93	7.25	2.87	6.44	7.77	3.96	3.02	5.39	-	-

These tracks are: Japan, Bahrain, Azerbaijan, Great Britain, Hungary, Italy, Singapore, Qatar, Abu Dhabi, Spain, Saudi Arabia, Austria, Mexico, and USA. The underlined results represent tracks in each of the RSRL models' training data, and the bold font highlights the best-performing model for each track

Table 4 Comparison of each RSRL-X model with the Fixed Strategy and the Mercedes Probabilistic models on their respective seen and unseen tracks as depicted in Table 3, with RSRL gains (losses) indicated with **Bold** (Italic, respectively) text

Models	All Tracks		Seen tracks		Unseen tracks	
	vs. FS	vs. MP	vs. FS	vs. MP	vs. FS	vs. MP
RSRL-1	+2.68	+3.60	−1.97	−1.77	+3.04	+6.40
RSRL-2	+4.44	+5.36	−0.92	−0.93	+4.44	+4.01
RSRL-9	−0.43	+0.49	−0.20	+0.87	−0.86	−0.21

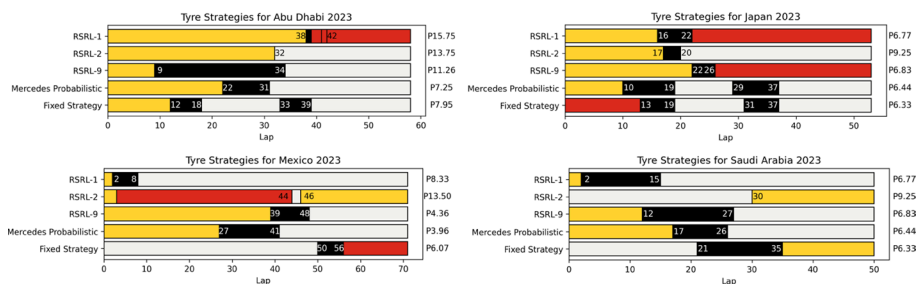


Fig. 7 The most common tyre strategies generated by each generalisability model, the Mercedes Probabilistic model, and two of the fixed strategies from the Fixed Strategy model for the Abu Dhabi, Japan, Mexico and Saudi Arabia GPs. The red, yellow and white regions represent the laps spent on the Soft, Medium and Hard tyres respectively. The black section represents an average period in which a pitstop is executed. This is evaluated by taking all simulations sharing the same strategy, and calculating the interquartile range for each pitstop

5 Explanations

In this section, we examine the quality of the deployed post-hoc XAI techniques by studying their fidelity and comprehensibility (§5). RSRL incorporates three such techniques to gain better insights into the RL models' behaviours, namely [TimeSHAP](#), [VIPER](#) and [decision tree counterfactuals](#). To evaluate the efficacy of the explanations, we undertake (high-level) assessments of their *fidelity*, i.e. how closely the explanations replicate the models, and *comprehensibility*, i.e. how comprehensible the explanations are to users. To demonstrate the use of the explanations in practice, we also exemplify how each of the explanations highlights the motivations behind the decisions of RSRL (from §4.2) in a race simulation. To do so, we analyse RSRL's decision of *no pit* on lap 10 of the 2023 Bahrain GP. While this single-track analysis does not cover the full scope of RSRL's varying behaviour across the circuits (which we leave to future work), it provides a comparative assessment of the explanations in one simulation setting at a crucial moment when pitstops are likely being considered.

TimeSHAP Feature Importance. To determine the fidelity of the feature importance approximator, we measure the Mean Absolute Error (MAE) for each feature over 1000 timesteps across 100 simulations. RSRL achieved an average error of 124.39. Normalised with the interquartile range of rewards observed (866.5), this represents a 14% normalised MAE, demonstrating that the TimeSHAP importance values by the approximator are mostly accurate and consistent with the actual rewards, thus reducing the likelihood of large errors when providing explanations for RSRL's actions.

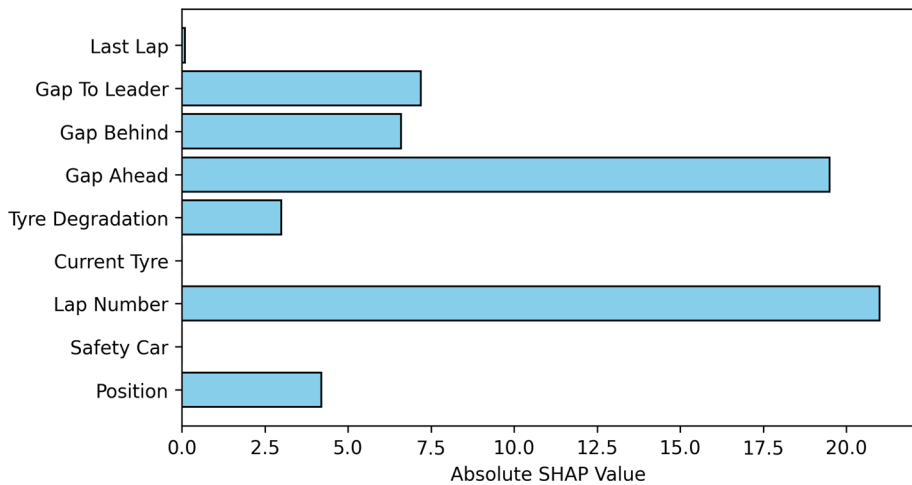


Fig. 8 The feature importance for RSRL on lap 10 of the 2023 Bahrain GP¹³

Table 5 Confusion matrix for the VIPER-trained decision tree of the best-trained model for predicted labels: *no pit*, *pit soft*, *pit medium* and *pit hard*

		Predicted Label			
		np	ps	pm	ph
True Label	np	4699	9	10	62
	ps	55	0	1	2
	pm	55	0	6	0
	ph	180	4	0	39

Feature importance is fairly simple to comprehend, whereby higher values are considered more important to a model's predictions. Even without explicit SHAP values, it is trivial to understand the importance of each feature to the other features (Jalali et al., 2023).

A feature importance plot for lap 10 of the 2023 Bahrain GP is presented in Fig. 8 indicating how RSRL is looking towards the car ahead, evidenced by the dominant Gap Ahead. The Lap Number plot is the most influential, indicating how RSRL is looking towards its expected pit window since observing the lap number informs RSRL of when to pit.¹³

VIPER Decision Tree Surrogate Model. The fidelity of the VIPER decision tree surrogate model is demonstrated with a confusion matrix generated from 100 random simulations, as seen in Table 5. The overall accuracy is 92.6%, however, the dataset is highly imbalanced with 93.5% of instances corresponding to the *no pit* class. Consequently, aggregate metrics are dominated by this majority. To provide a more balanced evaluation, we look at per-class and macro-averaged F1 scores. For the *no pit* class, we derive an F1 score of 0.962, whereas performance on decisions to pitstop is significantly lower: 0.000 for *pit soft*, 0.154 for *pit medium* and 0.240 for *pit hard*. This yields a macro-averaged F1 score of 0.339. Recall for the pit-stop actions are also very low which indicates that the surrogate failed to capture the strategically critical decisions: when the true label is a pitstop decision. While the surrogate model provides a high-fidelity approximation for the dominant *no pit*

¹³ These feature names have been reverted from their scaled counterparts that are passed into the model. For example, Gap Ahead is displayed, compared to Scaled Gap Ahead which is used by the model. These feature names have been swapped for clarity.

Fig. 9 Accuracy during training and testing of 50 iterations of VIPER decision trees. Each iteration covers all previous data points queried from the oracle (the best RSRL model), up to 26,714 in iteration 50

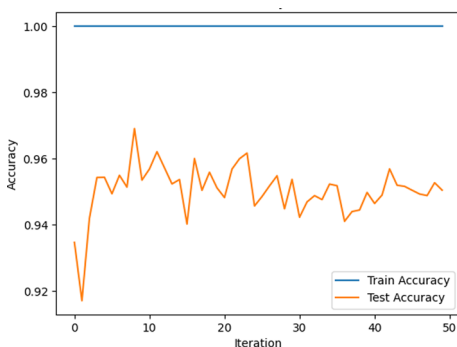
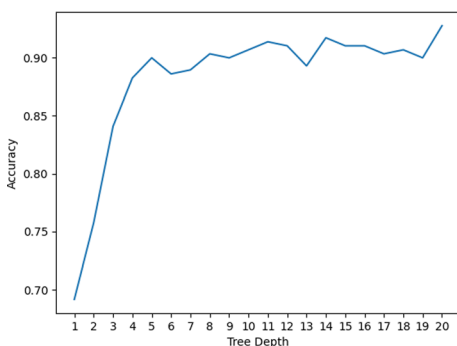


Fig. 10 Accuracy of VIPER decision tree testing with different maximum node depths



behaviour, its utility for analysing pit-stop strategies is limited. We leave to future work mitigations strategies which may improve these explanations' usefulness for race strategists, which may themselves have other issues, e.g. training a deeper decision tree would better capture these pitstop decisions, but would yield a much less explainable feature set. Figure 9 depicts the change in testing accuracy of the VIPER model over 50 iterations during training. A peak testing accuracy of 0.97 is achieved against the oracle; however, this metric is similarly influenced by class imbalance and should be interpreted in conjunction with the per-class results above.

Trained decision trees vary in comprehensibility depending on their depth. Bastani et al. (2018) generate trees with hundreds of nodes which becomes very difficult to comprehend. From depth testing the tree, we found that a maximum depth of 4 to 6 maintains good accuracy and looks to be sufficiently comprehensible. Figure 10 demonstrates how we reach a plateau around this range. However, with a larger input state space, this value is likely to increase as the tree has to split the dataset even further to achieve high accuracy.

Looking at the decision path for the *no pit* decision on lap 10 of the 2023 Bahrain GP (Table 6), we see that predictions are influenced by the tyres, race progress and the gaps. Similarly to the feature importance, we see that the race progress being less than 26% (15 laps) is an important factor. The first node decision on the current fitted tyre indicates how the model has different strategies based on the starting tyre, which is randomly chosen by the race simulator. The last lap to reference, s_{llr} feature value being ≤ 1.016 indicates that

Table 6 Decision tree path of the VIPER decision tree for RSRL on lap 10 of the 2023 Bahrain GP, resulting in its predicted *no pit* decision

Formal definition	Natural language translation
$\neg(s_{tyre} = \text{Hard})$	No Hard
$s_{td} \leq 2.560$	*Tyre Degradation ≤ 2.560
$s_{rp} \leq 0.263$	Race Progress ≤ 0.263
$s_{gl} \leq 35.550$	*Gap To Leader ≤ 35.550
$s_{ga} > 4.560$	*Gap Ahead > 4.560
$s_{gb} > 1.740$	*Gap Behind > 1.740
$s_{gb} > 1.800$	*Gap Behind > 1.800
$s_{llr} \leq 1.016$	Last Lap To Reference ≤ 1.016
$s_{gb} \leq 6.390$	*Gap Behind ≤ 6.390
$s_{td} > 0.860$	*Tyre Degradation > 0.860
$s_{gb} > 0.300$	*Gap Behind > 0.300

Features with * have had their scaling inversed for clarity¹⁴

the decision tree is also looking at the lap times, making its decision based on whether or not the current tyres begin slowing down significantly.¹⁴

In summary, these explanations may have promise in this setting, but further work is needed on improving their accuracy, in particular in capturing the pitstop decisions, before they could be useful in real-world deployments of RSRL.

Decision Tree Counterfactuals. To measure the fidelity of counterfactuals, we examine the proximity of generated counterfactuals to the original instance by counting the number of feature changes required to reach alternate decisions. By changing fewer features, the counterfactual demonstrates high fidelity because it suggests that the counterfactual closely aligns with the model's decision boundaries and feature importance. Taking 100 random simulations, and computing the closest counterfactual for a random alternate action, the average number of features requiring changing for the closest counterfactual was 1.630 whilst the average distance was 0.069. These low figures represent how these counterfactuals are sufficiently close to the real prediction, only requiring small but critical changes to result in a different action. This is in line with the criteria for good counterfactuals of Dandl et al. (2020), whereby they must be diverse and contain likely features.

Decision tree counterfactuals are presented in a textual format, listing exactly what changes are required to predict the alternate action. By demonstrating to the user exactly what changes are required, we believe that they are reasonably comprehensible to users but leave to future work a real-world user study with the race strategists to provide more evidence for this.

On lap 10 of the 2023 Bahrain GP, the explanations show what it would require to take an early pitstop, i.e. converting to a strategy with an extra pitstop. Looking at a hypothetical *pit soft* decision, the counterfactual tells us that we need to complete 4.503 more laps. Interestingly, this would take us to lap 14, the lap on which the (real-world) race winner Max Verstappen pitted for soft tyres in the 2023 Bahrain GP.

¹⁴These feature names have been reverted to their unscaled counterparts for clarity and a better understanding of the model's decisions. For example, Gap To Leader > 35.550 means that the decision at this node was taken based on whether or not the leader was more than 35.550 s ahead.

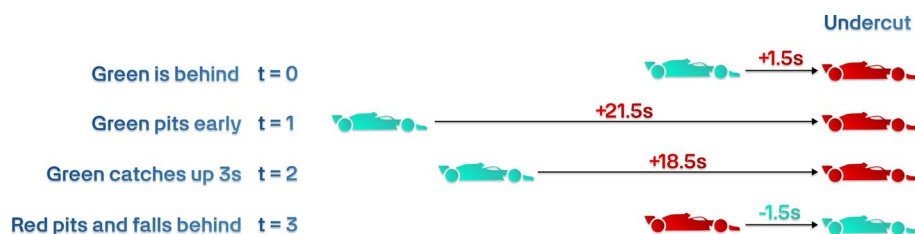


Fig. 11 Demonstration of how an undercut plays out over four sequential timesteps (points in the time during the race), where the loss in time due to a pitstop is 20s. Due to the difference in tyre degradation, the green car catches up to the red car after its pitstop. Subsequently, when the red car pits, it loses position to the green car

6 A Case Study with Emergent Tactics

Another important consideration is how well RSRL is able to draw parallels to strategic decisions taken by race strategists in real-world situations. We have already seen in §4 that the best checkpointed DQN model found a Soft-Hard-Hard strategy that almost perfectly mimicked Hamilton's tyre strategy employed in the 2023 Bahrain GP, demonstrated in Fig. 4. This phenomenon demonstrates how RL models can emulate the behaviour of traditional strategies through experience. The fact that RSRL further suggested the unconventional, yet optimal in simulation, Soft-Hard strategy demonstrates how race strategists could feasibly learn from RSRL itself. However, RSRL further extends this potential by demonstrating *emergent tactics*, i.e. race tactics exhibited as emergent behaviours of RSRL. We validate these race tactics, which are more fine-grained decisions than the overall strategy, through careful analysis of the explanations in a race simulation.

To demonstrate how RSRL exhibits emergent tactics, we focus on *undercutting*. This tactic, illustrated in Fig. 11, is one of the primary means for race strategists to facilitate their car overtaking a car in the position ahead, especially when degradation is high, and the time differential between a new and worn tyre is large. This comprises the undercutting car taking a pitstop earlier than the car ahead to take advantage of the relative, temporary advantage in pace due to tyre degradation. Although the overall race time may move away from that of the optimal strategy of the race overall, in this local period of the race, more time will be spent on newer tyres, giving faster lap times and thus potentially allowing the car to emerge ahead of its competitor after they complete their pitstop. On tracks where overtaking is difficult, the importance of a car's position can thus outweigh the effect of more degraded tyres.¹⁵

In this case study, we follow a simulation where Hamilton, controlled by RSRL, is starting in P6 at the 2023 Bahrain GP. We will focus particularly on two key decisions taken during the race, involving competitors Alonso (starting in P1) and Pérez (starting in P2). It should be noted that the Bahrain track, known for its high tyre degradation, presents ample opportunity for undercutting. Figure 12 plots the gap to the leader (s_{gl}) at each lap for each

¹⁵ It should be noted that the undercut is not always successful as this can result in the undercutting car's tyres becoming slower towards the end of the race, leaving it susceptible to overtakes. Another such race tactic which takes advantage of this effect at the end of the race is an *overtake* in which a car pitstops later than would be expected. We leave analysis of whether RSRL takes advantage of this tactic to future work.

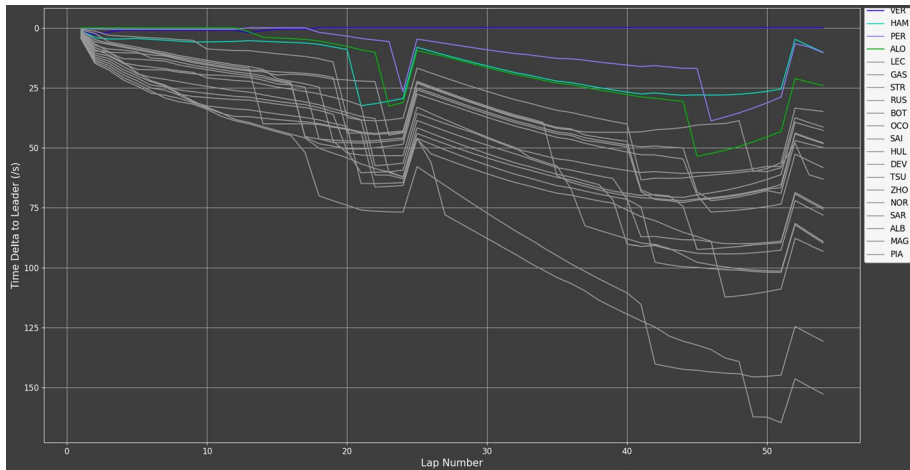


Fig. 12 The full race plot showing time delta to the leader, in seconds (on the y-axis), for each lap (x-axis); focusing on Hamilton (highlighted in light turquoise), Alonso (green) and Pérez (light purple). Verstappen is also included (dark blue) as a reference for the leader from lap 18 onwards. The legend denotes the final finishing positions of the race simulation and all other cars are greyed out for clarity

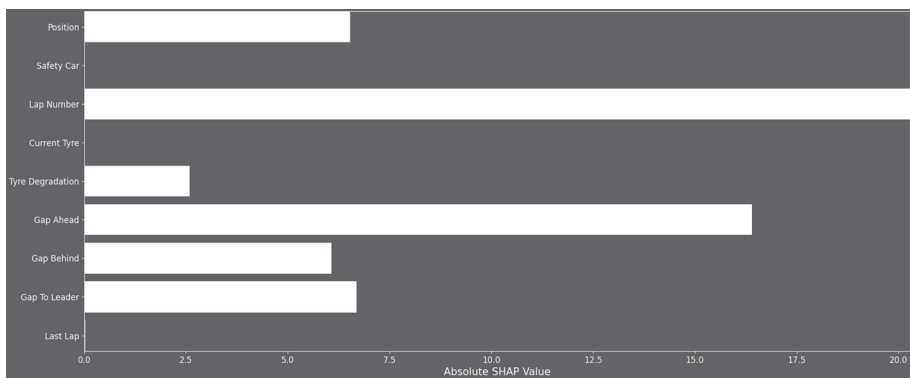


Fig. 13 The feature importance plot for Hamilton on lap 20 highlights how the gap to the driver ahead, Alonso, (s_{ga}) is almost as important as the lap number (s_{rp}) to RSRL, in deciding on whether or not to take a pitstop

car across the full race distance. Verstappen is also included to highlight the leader beyond lap 18 (the role, before which, both Alonso and Pérez fulfilled).

6.1 Hamilton's Lap 20 Undercut on Alonso

Let us focus on the battle between Hamilton, in P4 and Alonso, in P3, between laps 18 to 25. Looking at RSRL's feature importance plot on lap 20 in Fig. 13, we see that RSRL, in controlling Hamilton, is increasingly looking at the gap to the driver ahead (s_{ga}), Alonso,

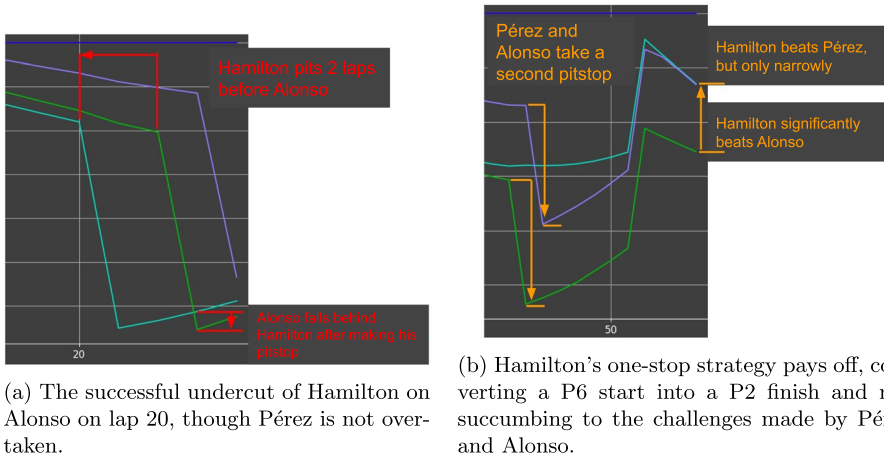


Fig. 14 Zoomed versions of Fig. 12, showing the gap to leader, Verstappen (dark blue), for Hamilton (turquoise), Alonso (green) and Pérez (purple). Note that the other drivers on whom we are not focusing have been removed for clarity

seemingly weighing up the pros and cons of an undercut as it approaches its optimal stop lap, indicated by the high importance on the Lap Number (s_{rp}) feature.¹⁶

Figure 14a shows the race plot at the point of interest, around lap 20. We see that just before Hamilton pits, both Hamilton and Alonso's gaps to the leader (Verstappen) start to increase, indicating that the tyre is starting to degrade significantly and is becoming considerably slower for them. Thus, it can be seen that the undercut looks like a suitable option for Hamilton in this scenario. Hamilton pits on lap 20 and thus drops further behind the race leader, Verstappen, who does not pit. On lap 21, we see how Hamilton, on newer and faster tyres, closes his gap to Verstappen, whilst Alonso falls further behind him. The narrowing gap between Alonso and Hamilton's lines indicates that the gap in time between them is decreasing. Finally, when Alonso pits on lap 22, he falls behind Hamilton, completing Hamilton's successful undercut.

6.2 Hamilton's Extended Hard Tyre Stint

Between laps 24 and 36, Hamilton is on a long hard tyre stint. Both Pérez and Alonso take a second pitstop around laps 43 and 44. They subsequently cede P2 to Hamilton whose pace thwarts any attempt by Pérez and Alonso to reclaim P2, as he holds this position until the end of the race (Fig. 14b).

Using the Decision tree counterfactuals, we can determine what changes to the race state would have been required for RSRL to take another pitstop on lap 44 to defend against

¹⁶ Recall from Table 1 that the model was not provided with a Lap Number feature, but a Race Progress, s_{rp} , feature instead. Since Race Progress is derived from the lap number, and indeed the model calculates feature importance on the Race Progress feature, we alter the name to Race Progress for clarity here (and similarly for the scaled features).

Alonso's pitstop onto faster tyres at lap 43. The closest *pit hard* decision node from the surrogate trained VIPER decision tree showed that the only change required was for Hamilton to be at least 10.785s closer to the car ahead, increasing the gap behind to 23.372s, i.e. a gap large enough to take a pitstop and still come out ahead. With such a large gap to the car behind, a second pitstop is a safe option due to the equal tyre degradation to the car behind and the guarantee that you will emerge ahead of it.

To further motivate RSRL's unconventional decision to stay out, we can force the simulation to deviate from RSRL's preferred action of *no pit* to *pit hard* on laps 43, 44, 45 and 46 to consider the possible alternate outcomes of taking another pitstop instead of staying out. Figure 15 demonstrates these four race outcomes.

Figure 15, reveals how in each case, Hamilton finishes P3 or P4 compared to the P2 achieved by not taking a pitstop and staying out on the hard tyre. Thus, taking a second pitstop, either in anticipation or in reaction to Alonso and Pérez, would have yielded a lower finishing position.

Overall, this case study presents a preliminary demonstration of how RSRL is not only able to optimise race strategy, but also presents emergent behaviours which model complex race tactics undertaken in the real world by race strategists. We believe that this merely touches on the potential of RSRL to both model the decisions taken in F1 race strategy but also potentially highlight novel strategies and tactics which could feasibly aid F1 race strategists in making decisions both before a race and as a live race support tool.

7 Conclusions and Future Work

In this paper, we have introduced RSRL, the first approach, to the best of our knowledge, to introduce RL supplemented with post-hoc XAI techniques to the domain of F1 Race Strategy. We have undertaken a wide-ranging evaluation of RSRL, demonstrating its promise in this fertile research area. We show that RSRL outperforms the baselines at selecting winning strategies both when focused on a single track (§4.2) and in a generalisability study across

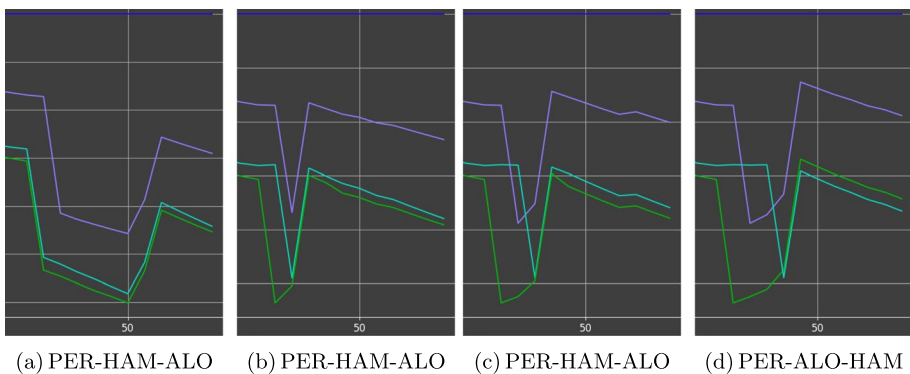


Fig. 15 The resulting races, with finishing positions indicated in the captions, if Hamilton (turquoise) takes a *pit hard* decision on laps 43 (15a), 44 (15b), 45 (15c) and 46 (15d), spanning the range of laps before and after both Alonso (green) and Pérez (purple) take their pitstops

multiple tracks (§4.3). We also demonstrated in a case study (§6) that not only is RSRL capable of learning winning strategies, but it also exhibits emergent tactics, i.e. fine-grained decisions that race strategists use to gain positions in the real world. The evidence for these findings was supported by the deployment of post-hoc XAI methods (§5), which helped to foster trust in RSRL even in the model development stage. Drawing parallels between the simulations and real-world phenomena with these explanations indicates how these models are deployable in practice. We also highlight how this work extends beyond F1, providing a framework for implementing portable and extendable RL systems through an abstraction architecture and deploying methods for explaining RSRL's decisions in real-time strategy applications.

We note that there are limitations in our work. We train our models with data obtained in a race simulator, which is a simplification of real-world scenarios. For example, the model may not have sufficient data to learn what to do when a safety car is deployed midway through the race, which rarely happens in the simulations. Such factors can limit the exploration of the large state space during model training and undermine the model performance. These issues could potentially be eliminated by using more data from real-world races, or more complex race simulations.

This work opens up numerous avenues for future work, not least a deeper exploration of the emergent tactics exhibited by RSRL, whether they are common tactics like the overcut, or indeed other potential tactics which are not used in the real world, which would give an opportunity for race strategists to learn from RSRL. Relatedly, we would like to leverage multi-agent RL to control multiple cars to explore cooperative strategies and tactics, e.g. splitting a teams' strategies to attempt both an undercut and an overcut on a car ahead. Further development of the reward function will also be targeted, with the aim of a more consistent and reliable set of strategies. This could be achieved by allowing for the injection of expert knowledge, such as candidate strategies or logical rules via reward shaping, e.g. as in (Gao et al., 2012). Additionally, we would like to improve strategy predictions based on a driver's driving style by fine-tuning models to suggest tailored and personalised strategies. Also, given that we have shown what can be achieved in learning these strategies from scratch, we would like to improve this process by incorporating human-in-the-loop feedback processes to utilise race strategists' priceless expertise, paving the way towards its deployment in live races. Finally, we would like to explore how our approach can be extended beyond F1, e.g. into other areas of motorsport, given the inherent similarities across these contexts.

Appendix A: Additional Experimental Results

Model Testing Results

The process for formulating the final RSRL model architecture began with the core specification. The four base architectures chosen were the DQN, CDQN, DRQN and A3C architectures, as explained in §4.1. Figure 16 demonstrates the average finishing positions for the four base models, checkpointed every 1000 simulations. The hyperparameters chosen for these models are detailed in Table 7. All experiments are formally defined [here](#).

Episodes	DQN	CDQN	DRQN	A3C
1,000	15.69	10.10	12.63	
2,000			8.67	
3,000		15.43	5.83	
4,000		13.91	6.73	
5,000	15.36	6.03	7.55	
6,000	6.31	7.10	6.43	9.83
7,000	6.00	8.17	5.27	9.83
8,000	11.70	16.47	5.67	9.83
9,000	5.79	14.14	8.55	9.93
10,000	15.96	14.97	6.27	8.10
11,000	8.19	15.43	6.67	9.93
12,000	9.48	14.53	5.30	9.93
13,000	6.68	15.67	6.57	9.93
14,000	5.72	11.55	5.31	10.28
15,000	5.93	12.44	5.10	10.28
16,000	6.30	12.21	5.47	10.28
17,000	6.00	13.97	5.27	9.00
18,000	11.79	12.64	5.03	10.28
19,000	7.34	14.59	5.47	10.28
20,000	5.86	11.08	5.60	
21,000	5.93	11.08	5.43	
22,000	5.22		4.90	
23,000	6.18		5.53	
24,000	6.54			
25,000	6.29			
Minimum	5.22	6.03	4.90	8.10
Top 5 Average	5.70	8.50	5.11	9.32

Fig. 16 The average finishing position for each 1000-simulation checkpoint for each of the DQN, CDQN, DRQN and A3C models

Table 7 Hyperparameters chosen for the DQN, CDQN, DRQN and A3C models

Hyperparameter	DQN	CDQN	DRQN	A3C
Epsilon	1	1	1	-
Epsilon Decay	0.999	0.999	0.995	-
Minimum Epsilon	0.005	0.005	0.01	-
Gamma	0.99	0.99	0.99	0.99
Learning Rate	0.001	0.001	0.001	0.001
Weight Decay	0.001	0.001	0.001	0.001
Replay Buffer Size	10,000	10,000	1000	-
Replay Buffer Sample Size	500	500	500	-
Episodes to Update Target	100	100	100	-
Value Loss Coefficient	-	-	-	0.5
Max Gradient Norm	-	-	-	50

A dash (–) indicates the hyperparameter is not applicable for that algorithm

Notable Hyperparameters Changed

Minimum Epsilon. We observed that reducing the minimum epsilon from 0.01 to 0.005 improved the average finishing position from P6.00 to **P5.39**. Our intuition is that, by reducing the minimum epsilon, the DRQN becomes more focused in terms of state space exploration towards the latter stages of training, allowing it to refine its strategy instead of taking sub-optimal actions.

Epsilon Decay. By increasing epsilon decay from 0.995 to 0.999, the average finishing position improved from P6.00 to **P5.20**. Similarly, when decreasing epsilon decay to 0.99, the model dropped in performance, achieving an average finish of P6.45. Increasing the epsilon decay could lead to more freedom for the model to explore different strategies in the early stages of training. We observe that the best environment for the model is to allow it to explore extensively in the early stages, but severely limit its exploration in the latter stages, to hone its strategy.

Architecture. From the architecture experiments, we found that the most effective changes were to increase the number of hidden layers and increase the size of each layer. Increasing from 64 to 128 neurons in each layer reduced the best model's average finishing position from P5.33 down to **P4.90**, whilst increasing the number of linear layers similarly decreased it to **P4.79**.

Hyperparameter Tuning Results

Having selected the DRQN as the base model, further hyperparameter tuning was completed manually, as described in §4.1. Figures 17 and 18 represent the various hyperparameter choices. Blue-highlighted cells represent changes to the base architecture. The results of each experiment can be seen in Figs. 19, 20 and 21.

Experiment Number	Epsilon	Epsilon Decay	Min Epsilon	Gamma	Learning Rate	Weight Decay	Replay Buffer Size	Episodes To Update Target
1	1	0.995	0.01	0.99	0.001	0.001	1000	100
2	1	0.99	0.01	0.99	0.001	0.001	1000	100
3	1	0.999	0.01	0.99	0.001	0.001	1000	100
4	1	0.995	0.02	0.99	0.001	0.001	1000	100
5	1	0.995	0.005	0.99	0.001	0.001	1000	100
6	1	0.995	0.01	0.98	0.001	0.001	1000	100
7	1	0.995	0.01	0.99	0.01	0.001	1000	100
8	1	0.995	0.01	0.99	0.0001	0.001	1000	100
9	1	0.995	0.01	0.99	0.001	0.001	100	100
10	1	0.995	0.01	0.99	0.001	0.001	1000	1000
11	1	0.995	0.01	0.99	0.001	0.001	1000	50
12	1	0.999	0.005	0.98	0.001	0.001	1000	100
13	1	0.999	0.005	0.98	0.001	0.001	100	100
14	1	0.999	0.005	0.99	0.001	0.001	1000	100
15	1	0.999	0.005	0.99	0.001	0.001	100	100
16	1	0.999	0.001	0.99	0.001	0.001	1000	100
17	1	0.999	0.001	0.99	0.001	0.001	100	100
18	1	0.999	0.001	0.98	0.001	0.001	1000	100
19	1	0.999	0.001	0.98	0.001	0.001	100	100
20	1	0.999	0.005	0.98	0.002	0.001	1000	100
21	1	0.999	0.005	0.98	0.002	0.001	100	100
22	0.8	0.999	0.005	0.98	0.001	0.001	1000	100
23	0.5	0.999	0.005	0.98	0.001	0.001	1000	100

Fig. 17 The DRQN hyperparameter selections for Experiments 1 to 23

Experiment Name	Architecture ID	Architecture	Notes (All run with Experiment 5)	L - Linear Layer R - ReLU E - ELU T - Tanh S - Sigmoid R6 - ReLU6
24	1	_L64 R L64 R LSTM(1, 128) L64 R L_	Regular model	
25	2	_L128 R L64 R LSTM(1, 256) L128 R L_	Increase size of each layer	
26	3	_L64 R L64 R LSTM(3, 128) L64 R L_	Increase LSTM layers	
27	4	_L64 R L64 R L64 R LSTM(1, 128) L64 R L64 L_	Increase hidden layers	
28	5	_L64 R L64 R GRU(1, 128) L64 R L_	GRU model	
29	6	_L64 R L64 R GRU(1, B, 128,) L64 R L_	Bidirectional GRU	
30	7	_L64 R L64 R GRU(3, 128,) L64 R L_	Increase GRU layers	
31	8	_L64 R L64 R RNN(1, 128) L64 R L_	RNN model	
32	9	_L64 R L64 R RNN(3, 128) L64 R L_	Increase RNN layers	
33	10	_LSTM(3, 128) L_	Lone LSTM	
34	11	_GRU(3, 128) L_	Lone GRU	
35	12	_RNN(3, 128) L_	Lone RNN	
(All run with Experiment 14)				
36	13	_L128 R L128 R L128 R LSTM(1, 256) L128 R L128 R L_	25 + 27	
37	14	_L128 E L128 E L128 E LSTM(1, 256) L128 E L128 E L_	36 + ELU Activations	
38	15	_L128 T L128 T L128 T LSTM(1, 256) L128 T L128 T L_	36 + Tanh Activations	
39	16	_L128 S L128 S L128 S LSTM(1, 256) L128 S L128 S L_	36 + Sigmoid Activations	
40	17	_L128 R6 L128 R6 L128 R6 LSTM(1, 256) L128 R6 L128 R6 L_	36 + ReLU6 Activations	
41	18	_L256 R L256 R L256 R LSTM(1, 512) L256 R L256 R L_	Larger 25 + 27	
42	19	_L512 R L512 R L512 R LSTM(1, 1024) L512 R L512 R L_	Largest 25 + 27	
43	20	_L512 R L512 R L512 R LSTM(1, 1024) L512 R L512 R L512 R L_	42 + Larger 27	

Fig. 18 The DRQN hyperparameter selections for Experiments 24 to 35

Episodes	Experiment 1	Experiment 2	Experiment 3	Experiment 4	Experiment 5	Experiment 6	Experiment 7	Experiment 8	Experiment 9	Experiment 10	Experiment 11
1,000	16.75	15.65		11.75	15.35	16.50	10.50		7.67	9.70	9.96
2,000	10.95	10.50		8.76	8.00	10.07	19.00		8.58	11.33	9.93
3,000	8.74	7.10	10.26	9.38	6.13	8.48	11.73		5.31	9.77	7.63
4,000	7.40	7.10	9.07	6.52	6.11	6.40	13.85		5.97	7.50	8.87
5,000	6.70	7.25	10.23	7.03	6.63	6.66	9.50	10.62	6.34	7.57	8.70
6,000	6.85	6.45	8.07	6.52	6.48	6.27	11.38	13.00	7.86	7.40	7.83
7,000	7.00	7.20	7.93	5.62	6.41	6.34	11.23	5.00	6.17	8.43	8.89
8,000	6.79	7.90	9.07	6.66	6.52	6.41	6.50	6.57	7.28	9.33	7.10
9,000	7.05	7.80	8.07	5.97	5.85	5.48	12.68	6.14	7.97	10.27	6.50
10,000	6.63	8.37	9.43	6.21	6.46	6.57	10.72	6.43	7.33	10.60	8.36
11,000	6.00	7.47	8.33	8.21	6.21	6.00	13.97		8.30	9.25	8.69
12,000	6.47	9.16	7.33	7.28	6.04	6.27	12.07		8.57	8.97	9.29
13,000	6.00	7.74	6.90	7.14	5.71	6.90	11.50		10.55	10.00	8.31
14,000	7.21	8.16	7.27	6.59	5.57	6.97	14.97		9.07	8.90	9.00
15,000	6.32	7.05	6.70	7.79	5.64	6.14	14.60		8.83	9.52	8.83
16,000	7.88	9.79	8.10	7.31	5.39	7.38	17.38		9.13	7.48	8.28
17,000	6.63	10.05	7.17	7.31	5.68	7.23	17.19		9.10	7.97	8.07
18,000	7.84	9.00	6.73	8.17	6.29	6.83	9.79		8.83	8.34	9.34
19,000	6.68	7.74	8.04	6.38	6.41	5.86	12.96		7.73	7.38	9.24
20,000	7.21	6.89	5.20	6.55	7.13	6.75	11.25		9.03	7.76	9.37
21,000	8.00	7.50	6.00	6.17	7.07	6.39	10.66		11.40	7.48	9.55
22,000	8.05	8.15	6.96	6.90	6.07	5.96	9.79		10.72	7.21	9.13
23,000	8.00		6.35	8.00	6.53	6.39				8.03	8.87
24,000			6.23	6.34							
25,000			6.48	5.36							
Minimum	6.00	6.45	5.20	5.36	5.39	5.48	8.50	5.00	5.31	7.21	6.50
Top 5 Average	6.28	6.92	6.05	5.87	5.60	5.89	9.62	6.95	6.21	7.39	7.43

Fig. 19 The average finishing position for each 1000-simulation checkpoint for DRQN hyperparameter Experiments 1 to 11

Episodes	Experiment 12	Experiment 13	Experiment 14	Experiment 15	Experiment 16	Experiment 17	Experiment 18	Experiment 19	Experiment 20	Experiment 21	Experiment 22	Experiment 23
1,000	14.75	16.32	14.86	15.93	15.29	14.27	15.44	13.75	16.96	16.48	14.90	18.05
2,000	15.66	15.14	12.93	9.30	9.83	12.83	15.14	12.60	12.93	15.77	12.34	11.87
3,000	13.17	15.47	11.03	12.10	12.17	9.03	14.86	8.67	12.20	9.57	11.10	14.07
4,000	9.86	15.70	10.14	7.30	14.07	7.83	15.44	8.64	10.07	6.64	12.68	6.52
5,000	6.67	12.83	6.60	8.03	6.24	8.87	9.63	4.89	9.38	16.45	14.00	6.00
6,000	6.25	11.67	6.26	7.10	5.72	8.28	9.97	6.85	8.32	16.30	7.43	7.04
7,000	7.82	7.87	5.56	6.47	5.52	8.03	9.61	6.07	9.14	11.30	7.59	7.89
8,000	8.04	14.07	6.37	5.43	6.24	5.33	8.60	5.04	7.61	11.87	7.60	7.57
9,000	7.89	17.97	5.22	6.63	6.03	6.97	8.73	5.04	6.89	15.68	7.75	8.57
10,000	8.61	9.67	4.85	5.57	5.46	5.41	7.93	6.48	6.64	13.82	6.83	7.82
11,000	7.66	8.63	5.33	7.30	5.41	5.31	7.77	7.48	7.04	15.04	7.07	6.67
12,000	8.11	8.17	4.81	5.53	5.03	5.17	7.69	7.11	7.07	13.75	6.59	7.04
13,000	7.54	9.13	5.93	7.17	6.28	5.77	7.69	6.11	7.00	11.33	8.14	7.14
14,000	5.85	11.30	5.11	7.20	5.29	7.83	6.36	5.30	5.93	8.04	5.21	
15,000	6.22	9.10	6.59	5.73	5.52	5.10	7.14	5.79	6.19	9.31	6.10	
16,000	7.39	10.70	5.22	5.73	5.37	5.53	6.83	5.96	5.32	7.64	5.90	
17,000	6.96	11.77	5.59	5.90	6.07	6.23	7.53	7.25	7.11	5.38	7.76	
18,000	7.36	10.60	5.30	7.00	5.76	5.73	7.47		6.71	7.59	5.66	
19,000		8.30										
20,000												
21,000												
22,000												
23,000												
24,000												
25,000												
Minimum	5.85	7.87	4.81	5.43	5.03	5.10	6.36	4.89	5.32	5.38	5.21	6.00
Top 5 Average	6.39	8.41	5.04	5.60	5.31	5.26	7.07	5.21	6.16	7.06	5.89	6.65

Fig. 20 The average finishing position for each 1000-simulation checkpoint for DRQN hyperparameter Experiments 12 to 23

Episodes	Experiment 24	Experiment 25	Experiment 26	Experiment 27	Experiment 28	Experiment 29	Experiment 30	Experiment 31	Experiment 32	Experiment 33	Experiment 34	Experiment 35
1,000	11.38	12.63	14.33	11.71	13.71	8.48	10.00				19.00	17.57
2,000	9.10	8.67	14.75	14.63	15.94	14.07	8.57	8.90	17.13	10.70	10.03	13.00
3,000	7.23	5.83	16.80	9.97	8.67	13.57	7.57	12.37	14.34	9.30	11.69	9.00
4,000	9.13	6.73	8.43	9.21	5.83	13.00	7.03	11.21	10.07	9.30	10.10	15.90
5,000	10.93	7.55	13.50	8.89	7.76	14.57	8.16	13.90	10.80	9.33	12.41	13.00
6,000	7.66	6.43	11.00	7.18	7.13	9.55	11.23	12.10	11.69	8.00	11.43	14.44
7,000	8.10	5.27	11.38	5.12	7.39	13.89	11.00	6.75	11.23	9.73	9.13	9.37
8,000	8.76	5.67	7.38	5.71	7.04	9.93	14.93	7.59	12.19	10.93	8.93	15.93
9,000	6.59	8.55	8.38	5.62	9.11	11.34	6.23	8.19	12.13	11.27	8.80	9.37
10,000	8.28	6.27	7.71	5.43	7.46	11.96	6.67	8.81	10.87	9.50	9.67	15.93
11,000	7.38	6.67	6.72	6.10	6.48	12.87	10.00	7.50		11.07	9.62	9.97
12,000	6.19	5.30	8.20	7.17	6.81	14.75	6.67	8.78	9.10	9.53	11.70	15.93
13,000	6.63	6.57	7.76	7.04	6.57	18.20	6.87	7.86	9.79	9.47	12.00	15.57
14,000	5.96	5.31	7.46	6.57	8.76	17.48	6.70	7.71	14.33	8.93	8.25	15.57
15,000	5.52	5.10	7.19	7.11	6.03	15.54	6.54	7.86		6.90	8.11	10.00
16,000	7.11	5.47	8.29	6.42	8.97	10.31	6.90	6.90	10.36	8.69	6.72	16.86
17,000	5.96	5.27	8.81	6.31	7.14	11.45	6.93	8.34	16.33	7.93	6.82	9.72
18,000	5.74	5.03	8.80	7.11	6.66	14.82	7.41	7.14	9.07	6.79	7.13	10.00
19,000	6.89	5.47	9.70	5.59	5.71	13.58	7.00	5.61	9.45	6.69	7.69	8.93
20,000	5.86	5.60	12.67	5.36	6.03	15.68	7.45	7.79	8.00	7.93	7.86	12.53
21,000	5.33	5.43	10.69	5.29	5.90	9.60	12.23	5.04	9.07	7.39	7.50	9.13
22,000	6.44	4.90	8.08	4.79	5.52	12.73	7.18	7.18		6.86	7.96	15.93
23,000		5.53		5.22	6.20	17.46		5.86			7.39	9.50
24,000						13.75					7.86	9.13
25,000												9.50
Minimum	5.33	4.90	6.72	4.79	5.52	8.48	6.23	5.04	8.00	6.69	6.72	8.93
Top 5 Average	5.68	5.11	7.29	5.16	5.80	9.57	6.56	6.03	8.94	6.93	7.11	9.11

Fig. 21 The average finishing position for each 1000-simulation checkpoint for DRQN hyperparameter Experiments 24 to 35

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s10994-026-07081-3>.

Acknowledgements The research described in this paper was partially supported by the EPSRC (grant number EP/X015823/1).

Author Contributions D.T. was the main author of the work, writing the first draft of the paper and developing the system. J.J. and A.K. provided technical expertise on the experiments on reinforcement learning and the explainable AI. A.Ru., S.W., S.S. and J.M. provided the technical expertise and system support from the Formula One side. F.B. was the secondary supervisor of the work, providing technical expertise on the reinforcement learning. A.Ra. was the main supervisor of the work, providing technical expertise on the explainable AI and writing the final draft of the paper. All authors reviewed the manuscript.

Data Availability The data supporting the findings of this study are proprietary and cannot be shared publicly due to confidentiality agreements and intellectual property restrictions. Access to the data may be provided upon request through appropriate channels, subject to approval and non-disclosure agreements with the relevant stakeholders.

Declarations

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Ali, S., Abuhmed, T., El-Sappagh, S. H. A., Muhammad, K., Alonso-Moral, J. M., Confalonieri, R., et al. (2023). Explainable artificial intelligence (XAI): What we know and what is left to attain trustworthy artificial intelligence. *Inf Fusion*, 99, Article 101805. <https://doi.org/10.1016/j.inffus.2023.101805>
- Bastani, O., Pu, Y., & Solar-Lezama, A. (2018). Verifiable reinforcement learning via policy extraction. In: NeurIPS. 2499–2509. <https://proceedings.neurips.cc/paper/2018/hash/e6d8545daa42d5ced125a4bf747b3688-Abstract.html>.
- Bekkemoen, Y. (2024). Explainable reinforcement learning (XRL): a systematic literature review and taxonomy. *Machine Learning*, 113(1), 355–441. <https://doi.org/10.1007/s10994-023-06479-7>
- Bento, J., Saleiro, P., Cruz, A. F., Figueiredo, M. A. T., & Bizarro, P. (2021). TimeSHAP: Explaining Recurrent Models through Sequence Perturbations. In: KDD 2565–2573. <https://doi.org/10.1145/3447548.3467166>.
- Berner, C., Brockman, G., Chan, B., Cheung, V., Debiak, P., & Dennison, C. (2019). Dota 2 with large scale deep reinforcement learning. CoRR, [arxiv: abs/1912.06680](https://arxiv.org/abs/1912.06680).
- Boettinger, M., & Klotz, D. (2023). Mastering Nordschleife - A comprehensive race simulation for AI strategy decision-making in motorsports. CoRR. <https://doi.org/10.48550/arXiv.2306.16088>, [arxiv: 2306.16088](https://arxiv.org/abs/2306.16088).
- Carreira-Perpiñán, M.Á., & Hada, S. S. (2021). Counterfactual explanations for oblique decision trees: Exact, efficient algorithms. In: AAAI 6903–6911. <https://doi.org/10.1609/aaai.v35i8.16851>.
- Dandl, S., Molnar, C., Binder, M., & Bischl, B. (2020). Multi-objective counterfactual explanations. In: PPSN 448–469. https://doi.org/10.1007/978-3-030-58112-1_31.
- Gao, Y., Toni, F., & Craven, R. (2012). Argumentation-based reinforcement learning for RoboCup soccer keepaway. In: ECAI 342–347. <https://doi.org/10.3233/978-1-61499-098-7-342>.
- Greydanus, S., Koul, A., Dodge, J., Fern, A. (2018). Visualizing and understanding atari agents. In: PMLR 1792–1801. <https://proceedings.mlr.press/v80/greydanus18a.html>.
- Hausknecht, M. J., & Stone, P. (2015). Deep recurrent Q-learning for partially observable MDPs. In: AAAI fall symposia. 29–37. <http://www.aaai.org/ocs/index.php/FSS/FSS15/paper/view/11673>.
- Hayes, B., & Shah, J. A. (2017). Improving robot controller transparency through autonomous policy explanation. In: HRI, pp. 303–312. <https://doi.org/10.1145/2909824.3020233>.
- Heilmeyer, A., Thomaser, A., Graf, M., & Betz, J. (2020). Virtual strategy engineer: Using artificial neural networks for making race strategy decisions in circuit motorsport. *Applied Sciences*. <https://api.semanticscholar.org/CorpusID:228907155>.
- Heine, O. F. C., & Thraves, C. (2023). On the optimization of pit stop strategies via dynamic programming. *Central European Journal of Operations Research*, 31(1), 239–268. <https://doi.org/10.1007/s10100-022-00806-4>
- Jalali, A. N., Haslhofer, B., Kriglstein, S., & Rauber, A. (2023). Predictability and comprehensibility in post-hoc XAI methods: A user-centered analysis. CoRR. [arxiv: 2309.11987](https://arxiv.org/abs/2309.11987). <https://doi.org/10.48550/arXiv.2309.11987>.
- Kalweit, G., Hügler, M., Werling, M., & Boedecker, J. (2020). Interpretable multi time-scale constraints in model-free deep reinforcement learning for autonomous driving. CoRR, [arxiv: abs/2003.09398](https://arxiv.org/abs/2003.09398).
- Liu, X., & Fotouhi, A. (2020). Formula-E race strategy development using artificial neural networks and Monte Carlo tree search. *Neural Computing and Applications*, 32(18), 15191–15207. <https://doi.org/10.1007/s00521-020-04871-1>
- Liu, X., Fotouhi, A., & Auger, D. J. (2021). Formula-E race strategy development using distributed policy gradient reinforcement learning. *Knowledge Based System*, 216, Article 106781. <https://doi.org/10.1016/j.knsys.2021.106781>
- Lundberg, S. M., & Lee, S. (2017). A unified approach to interpreting model predictions. In: NeurIPS; 4765–4774. <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>.
- Metz, L. D. (2013). Potential for passenger car energy recovery through the use of kinetic energy recovery systems (KERS). SAE 2013 World Congress & Exhibition. <https://doi.org/10.4271/2013-01-0407>,
- Milani, S., Topin, N., Veloso, M., & Fang, F. (2024). Explainable reinforcement learning: A survey and comparative review. *ACM Computing Surveys*, 56(7), 168. <https://doi.org/10.1145/3616864>
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T. P., Harley, T., et al. (2016). Asynchronous methods for deep reinforcement learning. In: ICML 1928–1937. <http://proceedings.mlr.press/v48/mnih16.html>.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., & Wierstra, D. (2013). Playing Atari with deep reinforcement learning. CoRR, [arxiv: abs/1312.5602](https://arxiv.org/abs/1312.5602).

- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533. <https://doi.org/10.1038/nature14236>
- Piccinotti, D., Likmeta, A., Brunello, N., & Restelli, M. (2021). Online planning for F1 race strategy identification. In: Bridging the gap between AI planning and reinforcement learning (PRL) – Workshop at ICAPS https://prl-theworkshop.github.io/prl2021/papers/PRL2021_paper_1.pdf.
- Ross, S., Gordon, G. J., & Bagnell, D. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. In: AISTATS 627–635. <http://proceedings.mlr.press/v15/ross11a/ross11a.pdf>.
- Shapley, L. S. (1953). A value for n-person games. In H. W. Kuhn & A. W. Tucker (Eds.), *Contributions to the theory of Games II* (pp. 307–317). Princeton University Press.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484–489. <https://doi.org/10.1038/nature16961>
- Skirzynski, J., Becker, F., & Lieder, F. (2021). Automatic discovery of interpretable planning strategies. *Machine Learning*, 110(9), 2641–2683. <https://doi.org/10.1007/s10994-021-05963-2>
- Thomas, D., Jiang, J., Kori, A., Russo, A., Winkler, S., Sale, S., et al. (2025). Explainable reinforcement learning for formula one race strategy. In: SAC. 1090–1097. <https://doi.org/10.1145/3672608.3707766>.
- van Seijen, H., Fatemi, M., Larocche, R., Romoff, J., Barnes, T., & Tsang, J. (2017). Hybrid reward architecture for reinforcement learning. In: NIPS. 5392–5402. <https://proceedings.neurips.cc/paper/2017/hash/1264a061d82a2edae1574b07249800d6-Abstract.html>.
- Verma, A., Murali, V., Singh, R., Kohli, P., & Chaudhuri, S. (2018). Programmatically interpretable reinforcement learning. In: ICML. 5052–5061. <http://proceedings.mlr.press/v80/verma18a.html>.
- Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., et al. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782), 350–354. <https://doi.org/10.1038/s41586-019-1724-z>
- Wang, Z., Veličković, P., Hennes, D., Tomašev, N., Prince, L., Kaisers, M., et al. (2024). TacticAI: an AI assistant for football tactics. *Nature Communications*, 15(1), 1906. <https://doi.org/10.1038/s41467-024-45965-x>
- Wurman, P. R., Barrett, S., Kawamoto, K., MacGlashan, J., Subramanian, K., Walsh, T. J., et al. (2022). Out-racing champion Gran Turismo drivers with deep reinforcement learning. *Nature*, 602(7896), 223–228. <https://doi.org/10.1038/s41586-021-04357-7>
- Zimmermann, A. (2023). Chance and the predictive limit in basketball (Both College and Professional). In: DS. 599–613. https://doi.org/10.1007/978-3-031-45275-8_40.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.